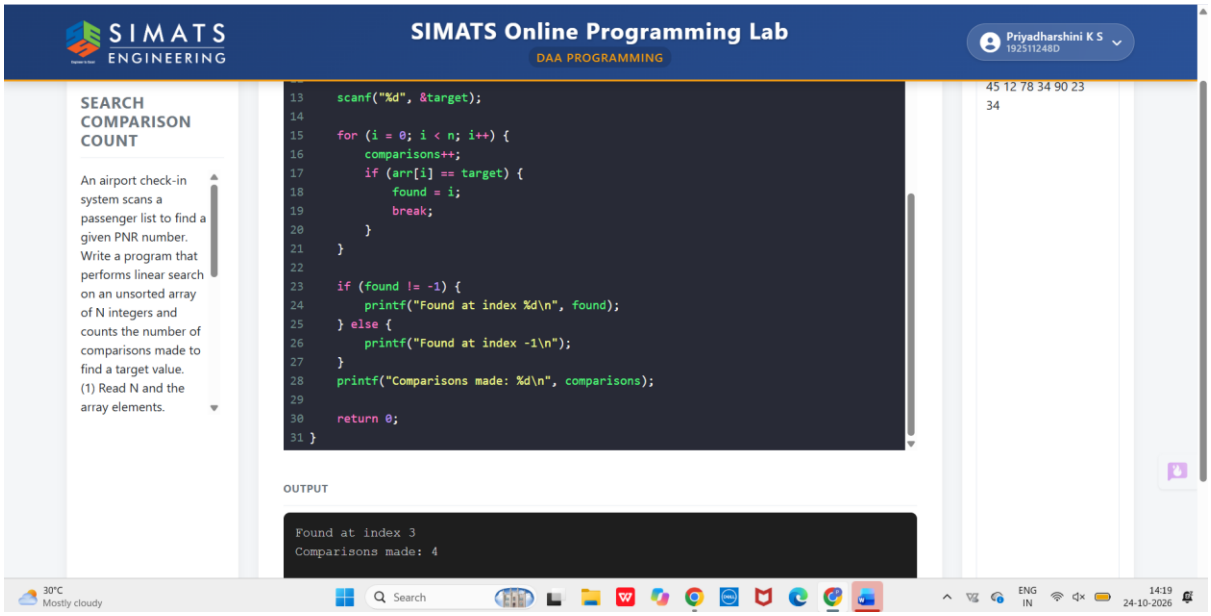
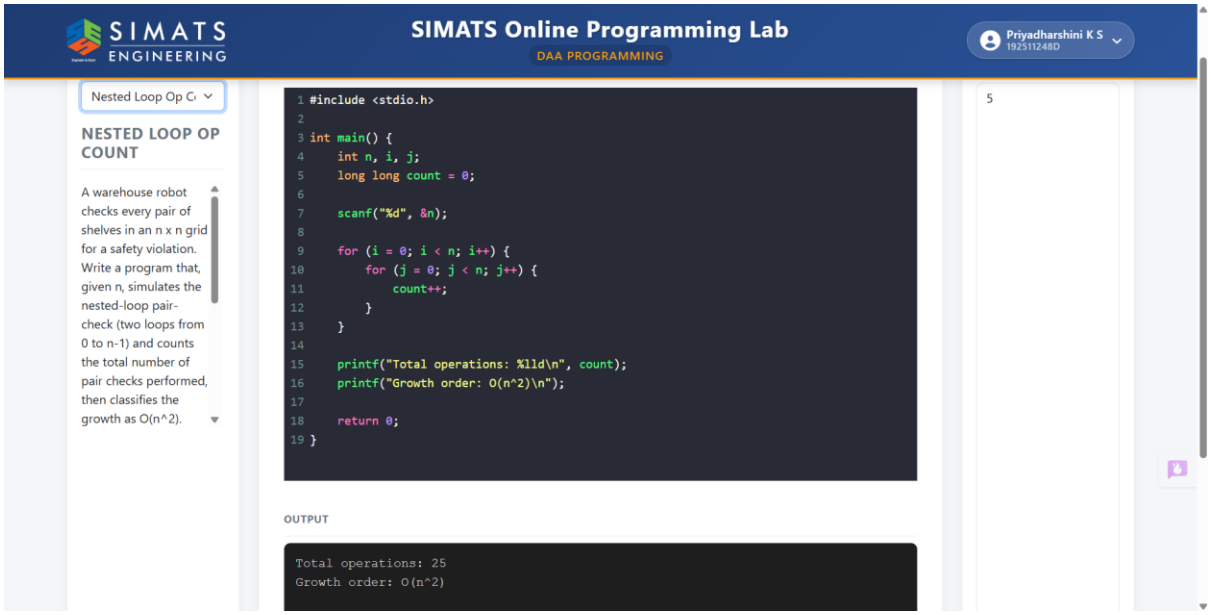
INTRODUCTION AND ANALYSIS





SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Priyadharshini K S
192511248D

Best Worst Avg Sea

BEST WORST AVG SEARCH

A ticket counter wants to know how search speed varies depending on where a visitor's token number sits in today's queue array. Write a program that performs linear search for three different targets - one at the first position, one at the last position, and one

```c
#include <stdio.h>

int linearSearch(int arr[], int n, int target, int *comparisons) {
    *comparisons = 0;
    for (int i = 0; i < n; i++) {
        (*comparisons)++;
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
int main() {
    int n;

    printf("Enter size of array: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
```

7 19 3 56 42

OUTPUT

```
Enter size of array: Enter 7 elements: Best Case (first element): 1 comparison
Worst Case (last element): 7 comparisons
Not Found Case: 7 comparisons
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Priyadharshini K S
192511248D

Factorial Growth Tr

FACTORIAL GROWTH TRACE

A statistics teacher wants students to see how the number of recursive calls grows with input size when computing factorials recursively. Write a program that computes n! recursively while counting the total number of recursive calls made.

```python

call_count = 0

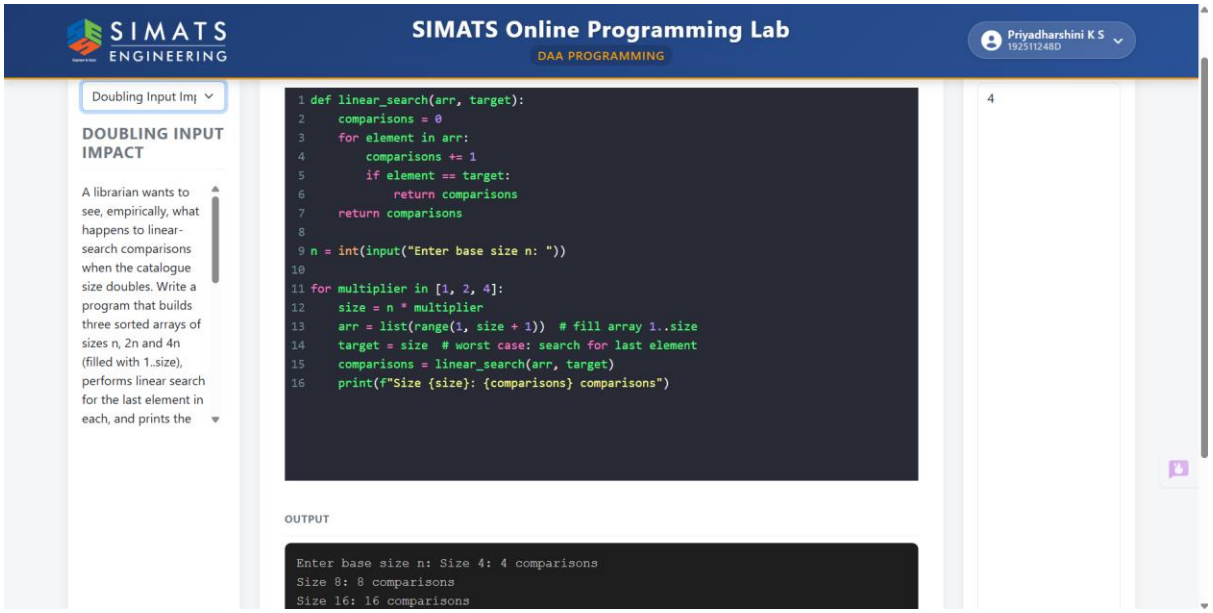
def factorial(n):
    global call_count
    call_count += 1
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

# Removing the prompt string to match standard competitive programming inputs
n = int(input())
result = factorial(n)
print(f"Factorial of {n} = {result}")
print(f"Total recursive calls: {call_count}")
```

6

OUTPUT

```
Factorial of 6 = 720
Total recursive calls: 6
```


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
Doubling Input Imp
DOUBLING INPUT IMPACT
A librarian wants to see, empirically, what happens to linear-search comparisons when the catalogue size doubles. Write a program that builds three sorted arrays of sizes n, 2n and 4n (filled with 1..size), performs linear search for the last element in each, and prints the
def linear_search(arr, target):
comparisons = 0
for element in arr:
comparisons += 1
if element == target:
return comparisons
return comparisons
n = int(input("Enter base size n: "))
for multiplier in [1, 2, 4]:
size = n * multiplier
arr = list(range(1, size + 1)) # fill array 1..size
target = size # worst case: search for last element
comparisons = linear_search(arr, target)
print(f"Size {size}: {comparisons} comparisons")
4
OUTPUT
Enter base size n: Size 4: 4 comparisons
Size 8: 8 comparisons
Size 16: 16 comparisons